Universidad de Granada

ETSIIT Informática y Telecomunicación

Dpto. de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Práctica 2

Técnicas de Búsqueda basadas en Poblaciones
para el Problema de Portfolio

Curso 2025–2026

Nombre: Jose Angel Carretero Montes
DNI: 25628892S
Email: joseangel1@correo.ugr.es
Grupo: Grupo 3
Horario: Martes 15:30–17:30

Índice

1. Descripción del Problema **2**

2. Aplicación de los Algoritmos al Problema **2**
2.1. Representación de soluciones 2
2.2. Generación aleatoria de soluciones 2
2.3. Operador de reparación (`fix`) 2
2.4. Operador de selección: torneo binario 3
2.5. Operadores de cruce 3
2.6. Operador de mutación 4

3. Algoritmos Genéticos y Meméticos **5**
3.1. Algoritmo Genético Generacional (AGG) 5
3.2. Algoritmo Genético Estacionario (AGE) 6
3.3. Algoritmos Meméticos (AM) 6

4. Estructura del Código y Manual de Usuario **8**

5. Experimentos y Resultados **8**
5.1. Casos del problema 8
5.2. Resultados 8
5.3. Boxplots 9

6. Análisis de Resultados **10**
6.1. Impacto del operador de cruce: AGG-Arit vs AGG-BLX 10
6.2. Impacto del operador de cruce: AGE-Arit vs AGE-BLX 11
6.3. Comparación de esquemas: AGG vs AGE 11
6.4. Algoritmos Meméticos: AM-All vs AM-Rand vs AM-Best 11
6.5. Búsqueda Local vs Algoritmos Genéticos 12
6.6. Capacidad de generalización 12
6.7. Variabilidad, coste computacional y análisis de boxplots 12
6.8. Conclusiones generales 13

1. Descripción del Problema

El **Problema de Portfolio** consiste en determinar la asignación óptima de capital entre N activos financieros con el objetivo de maximizar el retorno esperado minimizando simultáneamente el riesgo.

Representación de los datos

Para cada activo $i \in \{1, \ldots, N\}$ y día $t \in \{1, \ldots, T\}$ se dispone del retorno diario μ_{ti}, calculado como:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Los datos cubren el periodo 2015–2024 para entrenamiento y el año 2025 para evaluación.

Función objetivo

Dado un vector de pesos $w = (w_1, \ldots, w_N)$:

$$\text{fitness}(w) = \text{Beneficio}(w) - \lambda \cdot \text{Riesgo}(w)$$

$$\text{Beneficio}(w) = \sum_{i=1}^{N} w_i \cdot \sum_{t=1}^{T} \log(1 + \mu_{ti}) \qquad \text{Riesgo}(w) = \sqrt{\sum_{i=1}^{N} \sum_{j=1}^{N} w_i \cdot \Sigma_{ij} \cdot w_j}$$

siendo Σ_{ij} la covarianza entre activos i y j, y $\lambda = 500$.

Restricciones

$$\sum_{i=1}^{N} w_i = 1 \qquad w_i = 0 \ \vee \ lo \leq w_i \leq hi \quad \forall i$$

2. Aplicación de los Algoritmos al Problema

2.1. Representación de soluciones

Una solución es un vector $w \in \mathbb{R}^N$ donde w_i indica la fracción de capital invertida en el activo i: exactamente cero o en $[lo, hi]$.

2.2. Generación aleatoria de soluciones

Igual que en la Práctica 1: subconjunto aleatorio de activos activos, pesos aleatorios en $[lo, hi]$, normalización y reparación.

2.3. Operador de reparación (`fix`)

Tres fases: normalización, recorte de rango y redistribución iterativa. Es esencial tras los operadores de cruce, que pueden generar soluciones infactibles.

2.4. Operador de selección: torneo binario

Se eligen dos individuos al azar y se devuelve el de mayor fitness.

Algorithm 1: Torneo binario

Entrada: población P
Salida: individuo ganador
$i_1 \leftarrow$ aleatorio en $[0, |P|-1]$; $i_2 \leftarrow$ aleatorio en $[0, |P|-1]$;
si $P[i_1].\mathit{fitness} \geq P[i_2].\mathit{fitness}$ **entonces**
  devolver $P[i_1]$
devolver $P[i_2]$

2.5. Operadores de cruce

Cruce Aritmético

Para cada posición i se genera $r_i \in [0,1]$ independiente:

$$\text{Off}_i^1 = r_i \cdot S_i^1 + (1-r_i) \cdot S_i^2 \qquad \text{Off}_i^2 = r_i \cdot S_i^2 + (1-r_i) \cdot S_i^1$$

Algorithm 2: Cruce Aritmético(S^1, S^2, problema)

Entrada: padres S^1, S^2; $[lo, hi]$
Salida: hijos Off^1, Off^2
$\text{Off}^1 \leftarrow S^1$; $\text{Off}^2 \leftarrow S^2$;
para $i \leftarrow 1$ **to** N **hacer**
  $r \leftarrow$ aleatorio en $[0,1]$;
  $\text{Off}^1[i] \leftarrow r \cdot S^1[i] + (1-r) \cdot S^2[i]$;
  $\text{Off}^2[i] \leftarrow r \cdot S^2[i] + (1-r) \cdot S^1[i]$;
`fix(`Off^1`)`; `fix(`Off^2`)`; **devolver** Off^1, Off^2

Cruce BLX-α

Con $I_i = |S_i^1 - S_i^2|$:

$$hmin_i = \text{mín}(S_i^1, S_i^2) - \alpha \cdot I_i \qquad hmax_i = \text{máx}(S_i^1, S_i^2) + \alpha \cdot I_i$$

Ambos hijos se generan uniformemente en $[\text{máx}(0, hmin_i), \text{mín}(hi, hmax_i)]$, $\alpha = 0{,}3$.

Algorithm 3: Cruce BLX-α(S^1, S^2, problema), $\alpha = 0{,}3$

Entrada: padres S^1, S^2; $[lo, hi]$; α
Salida: hijos Off^1, Off^2

para $i \leftarrow 1$ **to** N **hacer**
  $I \leftarrow |S^1[i] - S^2[i]|$;
  $hmin \leftarrow \text{máx}(0,\ \text{mín}(S^1[i], S^2[i]) - \alpha \cdot I)$;
  $hmax \leftarrow \text{mín}(hi,\ \text{máx}(S^1[i], S^2[i]) + \alpha \cdot I)$;
  $\text{Off}^1[i] \leftarrow$ aleatorio en $[hmin, hmax]$;
  $\text{Off}^2[i] \leftarrow$ aleatorio en $[hmin, hmax]$;
`fix(`Off^1`)`; `fix(`Off^2`)`; **devolver** Off^1, Off^2

2.6. Operador de mutación

Para Algoritmos Genéticos

Con probabilidad $p_m = 0{,}1$ por individuo se selecciona un gen aleatorio y se reemplaza por un valor uniforme en $[lo, hi]$, seguido de `fix()`. Solo cuenta como evaluación si hay mutación.

Algorithm 4: Mutación AG(*ind*, problema)

si *rand()* $\leq 0{,}1$ **entonces**
  $gen \leftarrow$ aleatorio en $[0, N-1]$;
  $ind[gen] \leftarrow$ aleatorio en $[lo, hi]$;
  `fix(`*ind*`)`; $evals \leftarrow evals + 1$;

Para Algoritmos Meméticos

Con probabilidad $p_m = 0{,}1/N$ por gen se reemplaza independientemente. Solo cuenta como evaluación si se muta algún gen.

Algorithm 5: Mutación AM(*ind*, problema)

$mutado \leftarrow$ falso;
para $i \leftarrow 1$ **to** N **hacer**
  si *rand()* $\leq 0{,}1/N$ **entonces**
    $ind[i] \leftarrow$ aleatorio en $[lo, hi]$; $mutado \leftarrow$ verdadero;
si *mutado* **entonces**
  `fix(`*ind*`)`; $evals \leftarrow evals + 1$;

3. Algoritmos Genéticos y Meméticos

3.1. Algoritmo Genético Generacional (AGG)

Parámetros

$N_p = 100$, $p_c = 0{,}8$, $p_m = 0{,}1$ por individuo, criterio de parada: 10 000 evaluaciones.

Esquema de evolución y reemplazamiento

Algorithm 6: AGG (generacional con elitismo)

Entrada: problema, maxEvals
Salida: mejor solución

Inicializar P con N_p individuos aleatorios; $evals \leftarrow N_p$;
mientras $evals < maxEvals$ **hacer**
  $P' \leftarrow \emptyset$;
  para $i \leftarrow 1$ **to** $N_p/2$ ***y*** $evals < maxEvals$ **hacer**
    $p_1 \leftarrow$ `torneo(`P`)`; $p_2 \leftarrow$ `torneo(`P`)`;
    si $rand() \leq 0{,}8$ **entonces**
      $(h_1, h_2) \leftarrow$`cruzar(`p_1, p_2`)`; $evals$ += 2
    si no si $h_1 \leftarrow p_1$; $h_2 \leftarrow p_2$ **entonces**
      `mutar`
    h_1; `mutar(`h_2`)`; $P' \leftarrow P' \cup \{h_1, h_2\}$;
  si máx(P').*fitness* $<$ máx(P).*fitness* **entonces**
    mín$(P') \leftarrow$ máx(P)
  $P \leftarrow P'$;
devolver máx(P)

Se implementan dos variantes: **AGG-Arit** y **AGG-BLX**.

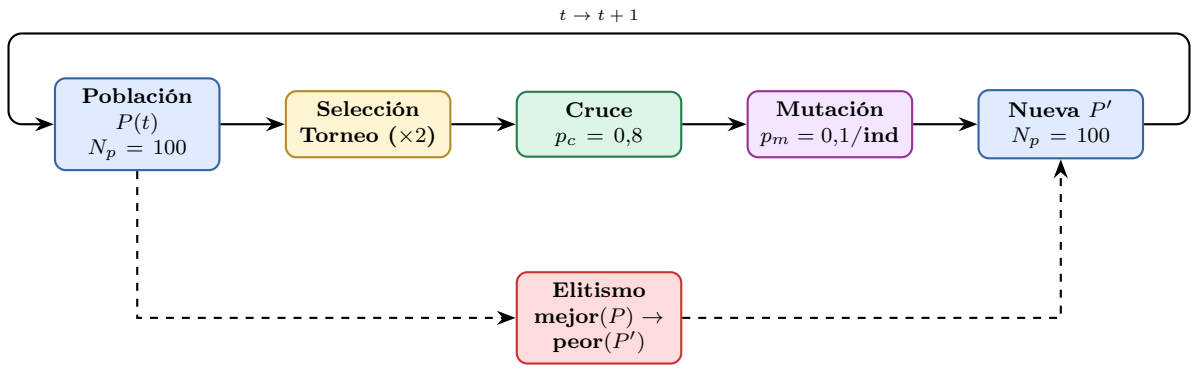


Figura 1: Ciclo evolutivo del AGG. Las líneas discontinuas muestran el flujo de elitismo.

3.2. Algoritmo Genético Estacionario (AGE)

Parámetros

$N_p = 100$, $p_c = 1{,}0$ (siempre se cruzan), $p_m = 0{,}1$ por individuo, criterio de parada: 10 000 evaluaciones.

Esquema de evolución y reemplazamiento

Algorithm 7: AGE (estacionario)

Entrada: problema, maxEvals
Salida: mejor solución

Inicializar P con N_p individuos aleatorios; $evals \leftarrow N_p$;
mientras $evals < maxEvals$ **hacer**
  $p_1 \leftarrow$`torneo(`P`)`; $p_2 \leftarrow$`torneo(`P`)`;
  $(h_1, h_2) \leftarrow$`cruzar(`p_1, p_2`)`; $evals$ += 2;
  `mutar(`h_1`)`; `mutar(`h_2`)`;
  si $h_1.fitness >$ mín$(P).fitness$ **entonces**
    mín$(P) \leftarrow h_1$
  si $h_2.fitness >$ mín$(P).fitness$ **entonces**
    mín$(P) \leftarrow h_2$
devolver máx(P)

Se implementan dos variantes: **AGE-Arit** y **AGE-BLX**.

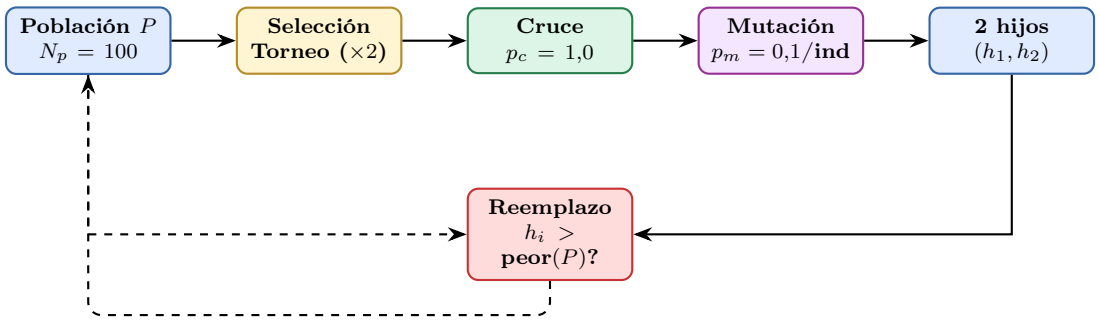


Figura 2: Ciclo evolutivo del AGE. Las flechas discontinuas muestran la consulta del peor individuo de P y la actualización en sitio: solo 0–2 miembros cambian por iteración.

3.3. Algoritmos Meméticos (AM)

Los AMs hibridan el AGG que mejor resultado ha proporcionado (**AGG-BLX**, véase Sección 5) con la BL de la Práctica 1 adaptada: fracción de movimiento $p = 0{,}15$ (en lugar de 0,40), máximo 100 evaluaciones por llamada. La BL se aplica **cada 10 generaciones**; el presupuesto de cada llamada es mín(100, maxEvals − $evals$) para respetar el límite global de 10 000 evaluaciones (incluidas las de la BL).

Parámetros comunes

$N_p = 100$, $p_c = 0{,}8$, $p_m = 0{,}1/N$ por gen.

AM-All: BL a todos los individuos

Algorithm 8: AM-All

```
si generaciones mód 10 = 0 entonces
    para i ← 1 to |P| y evals < maxEvals hacer
        budget ← mín(100, maxEvals − evals);
        P[i] ← BL(problem, P[i], budget); evals += bl.evals;
```

AM-Rand: BL aleatoria al 10 % de la población

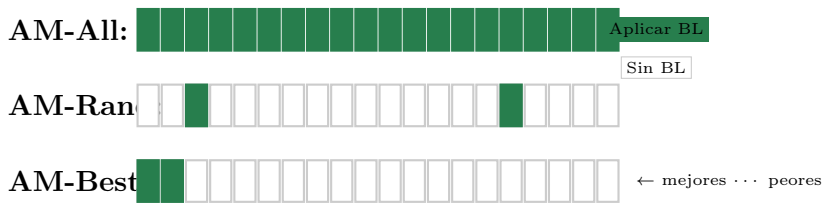
Algorithm 9: AM-Rand

```
si generaciones mód 10 = 0 entonces
    para i ← 1 to |P| y evals < maxEvals hacer
        si rand() ≤ 0,1 entonces
            budget ← mín(100, maxEvals − evals);
            P[i] ← BL(problem, P[i], budget); evals += bl.evals;
```

AM-Best: BL a los $0{,}1 \cdot N_p$ mejores

Algorithm 10: AM-Best

```
si generaciones mód 10 = 0 entonces
    ordenar P descendente por fitness;
    para i ← 1 to mín(0,1 · |P|, |P|) y evals < maxEvals hacer
        budget ← mín(100, maxEvals − evals);
        P[i] ← BL(problem, P[i], budget); evals += bl.evals;
```



(Representación a escala 1:5 de población N_p=100)

Figura 3: Comparación de estrategias de selección en AMs. Cada célula representa 5 individuos.

4. Estructura del Código y Manual de Usuario

```
portfolio/
  main.cpp          src/portfolioProblem.cpp, randomSearch.cpp
  CMakeLists.txt    src/greedy.cpp, localsearch.cpp, genetic.cpp
  data/             inc/portfolioProblem.h, randomSearch.h
    ibex_35.csv         greedy.h, localsearch.h, genetic.h
    syp_100.csv     common/mh.h, problem.h, random.hpp, util.h
    syp_500.csv
```

Compilación y ejecución:

```
cmake .  &&  make
./main [max_evals] [n_seeds]   # defecto: 10000 evals, 50 semillas
```

5. Experimentos y Resultados

5.1. Casos del problema

Fichero	N	Días train	Días test	$[lo, hi]$
ibex_35.csv	30	2559	254	[0,005, 0,08]
syp_100.csv	97	2516	249	[0,005, 0,05]
syp_500.csv	457	2516	249	[0,005, 0,02]

Parámetros comunes: $\lambda = 500$, MAX_EVALS= 10 000, semillas 1–50.

5.2. Resultados

Cuadro 1: IBEX 35 — $N = 30$, $[lo, hi] = [0{,}005, 0{,}08]$ (medias, 50 semillas)

Algoritmo	Fit.Train	Fit.2025	Ben.2025	Evals	Tiempo (s)
Greedy	−4,650	−3,387	0,115	1,0	0,000
Random	−4,941	−3,306	0,036	10000,0	0,024
BL	−4,491	−3,085	0,029	2115,8	0,001
AGG-Arit	−4,707	−3,156	0,041	10000,6	0,006
AGG-BLX	−4,559	−3,113	0,038	10000,6	0,008
AGE-Arit	−4,601	−3,119	0,035	10000,7	0,008
AGE-BLX	−4,521	−3,102	0,034	10000,7	0,008
AM-All	−4,865	−3,207	0,047	10000,0	0,025
AM-Rand	−4,516	−3,101	0,038	10000,2	0,016
AM-Best	−4,503	−3,098	0,037	10000,4	0,016

Cuadro 2: S&P 100 — $N = 97$, $[lo, hi] = [0{,}005, 0{,}05]$ (medias, 50 semillas)

Algoritmo	**Fit.Train**	**Fit.2025**	**Ben.2025**	**Evals**	**Tiempo (s)**
Greedy	−3,664	−5,020	0,010	1,0	0,000
Random	−3,390	−4,113	0,080	10000,0	0,064
BL	−2,791	−3,757	0,109	10000,0	0,039
AGG-Arit	−3,094	−3,830	0,087	10000,5	0,030
AGG-BLX	−3,074	−3,820	0,085	10000,5	0,037
AGE-Arit	−3,030	−3,807	0,089	10000,7	0,029
AGE-BLX	−2,968	−3,777	0,090	10000,5	0,034
AM-All	−3,410	−4,080	0,074	10000,0	0,221
AM-Rand	−3,112	−3,824	0,081	10000,2	0,140
AM-Best	−3,079	−3,809	0,086	10000,6	0,133

Cuadro 3: S&P 500 — $N = 457$, $[lo, hi] = [0{,}005, 0{,}02]$ (medias, 50 semillas)

Algoritmo	**Fit.Train**	**Fit.2025**	**Ben.2025**	**Evals**	**Tiempo (s)**
Greedy	−3,438	−5,205	−0,063	1,0	0,000
Random	−3,642	−4,430	0,012	10000,0	1,734
BL	−2,628	−3,916	0,023	10000,0	1,168
AGG-Arit	−2,972	−3,980	0,017	10000,5	0,656
AGG-BLX	−2,871	−3,924	0,019	10000,8	0,673
AGE-Arit	−2,969	−3,977	0,016	10000,6	0,654
AGE-BLX	−2,876	−3,935	0,018	10000,7	0,663
AM-All	−3,484	−4,288	0,016	10000,0	2,111
AM-Rand	−2,950	−3,927	0,022	10000,2	1,542
AM-Best	−2,933	−3,934	0,018	10000,6	1,326

5.3. Boxplots

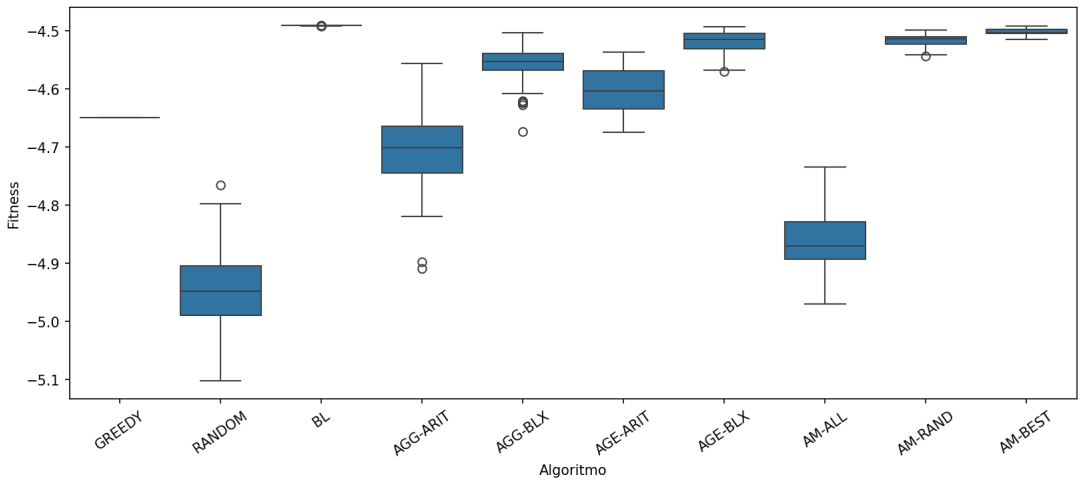


Figura 4: Distribución del fitness (2015–2024) en IBEX 35.

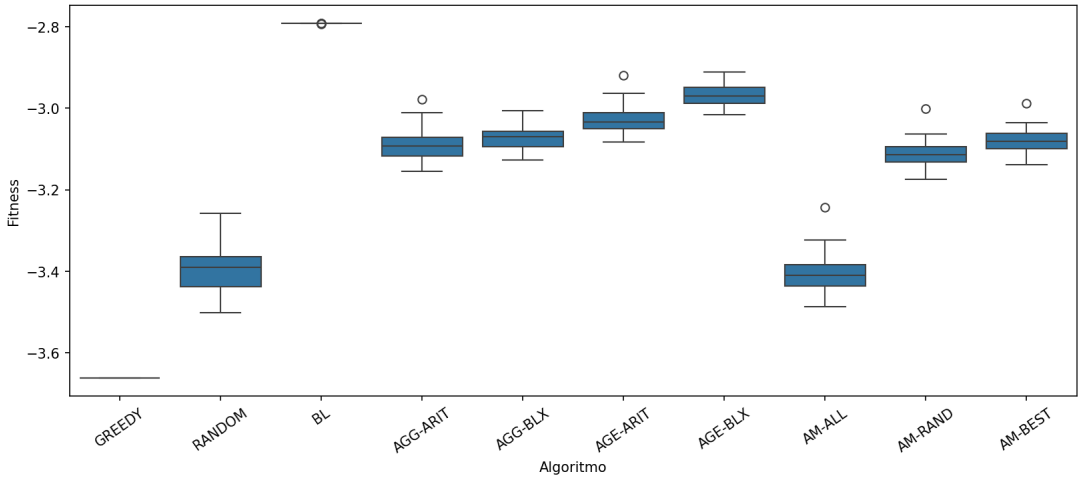


Figura 5: Distribución del fitness (2015–2024) en S&P 100.

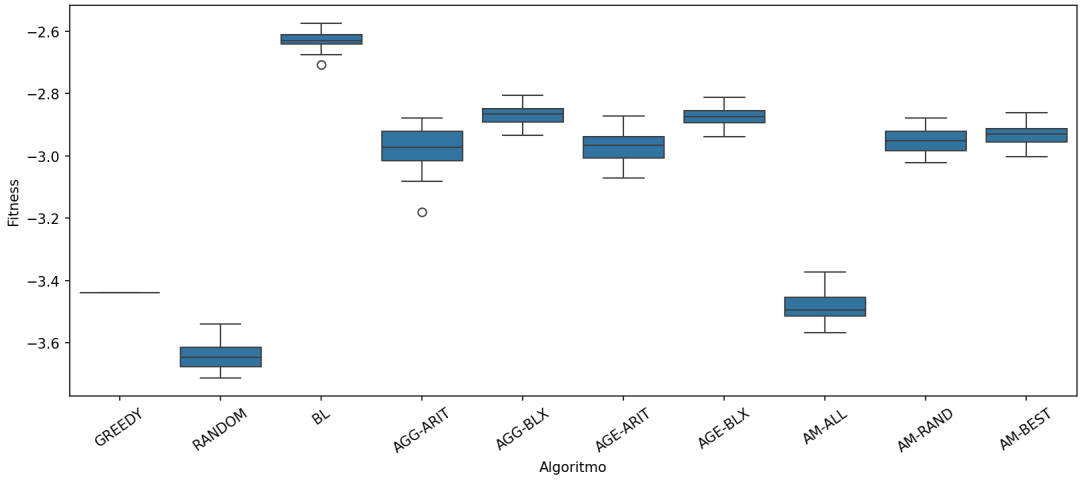


Figura 6: Distribución del fitness (2015–2024) en S&P 500.

6. Análisis de Resultados

6.1. Impacto del operador de cruce: AGG-Arit vs AGG-BLX

Dataset	AGG-Arit	AGG-BLX	Mejora BLX
ibex_35	−4,707	−4,559	0,148
syp_100	−3,094	−3,074	0,020
syp_500	−2,972	−2,871	0,101

BLX-α supera al Aritmético en los tres datasets. El cruce aritmético es puramente explotativo (genera hijos dentro del convex hull de los padres), mientras que BLX puede extenderse más allá, permitiendo explorar nuevas regiones. Esta ventaja es crítica tanto en espacios pequeños (mayor probabilidad de escape de óptimos locales) como grandes (mayor diversidad necesaria). Estos resultados justifican que los AMs usen **AGG-BLX** como base.

6.2. Impacto del operador de cruce: AGE-Arit vs AGE-BLX

Dataset	AGE-Arit	AGE-BLX	Mejora BLX
ibex_35	−4,601	−4,521	0,080
syp_100	−3,030	−2,968	0,062
syp_500	−2,969	−2,876	0,093

BLX mejora consistentemente AGE en todos los datasets, confirmando que la ventaja del operador de cruce es independiente del esquema de reemplazamiento.

6.3. Comparación de esquemas: AGG vs AGE

Dataset	AGG-BLX	AGE-BLX	Ganador
ibex_35	−4,559	−4,521	AGE (+0,038)
syp_100	−3,074	−2,968	AGE (+0,106)
syp_500	−2,871	−2,876	AGG (+0,005)

AGE supera a AGG en tamaño pequeño-medio; en syp_500 son prácticamente equivalentes (diferencia de 0,005). El AGG reemplaza toda la población cada generación, provocando *drift* genético pese al elitismo. El AGE mantiene 98 individuos intactos por iteración, conservando implícitamente una élite. Con presupuesto limitado (10000 evals), esta estrategia conservadora es superior. En problemas grandes, la exploración masiva del AGG (50 pares/generación vs. 2 en AGE) compensa el drift.

6.4. Algoritmos Meméticos: AM-All vs AM-Rand vs AM-Best

	ibex_35		syp_100		syp_500	
AM	Fit.Train	Evals	Fit.Train	Evals	Fit.Train	Evals
AM-All	−4,865	10000,0	−3,410	10000,0	−3,484	10000,0
AM-Rand	−4,516	10000,2	−3,112	10000,2	−2,950	10000,2
AM-Best	−4,503	10000,4	−3,079	10000,6	−2,933	10000,6

AM-All es el peor AM en todos los datasets, incluso inferior al mejor AG (ibex_35: AM-All −4,865 vs AGE-BLX −4,521, diferencia −0,344). La causa: AM-All aplica BL a ≈ 90 individuos en la primera fase memetica (generación 10), agotando casi todo el presupuesto con solo 10 generaciones de evolución previa. Los individuos inmaduros quedan atrapados en óptimos locales por la BL prematura.

AM-Best y AM-Rand obtienen resultados similares a los mejores AGs, con AM-Best ligeramente superior (ibex_35: −4,503 vs −4,516, diferencia +0,013): aplicar BL a los 10 mejores es más eficiente que seleccionarlos al azar.

6.5. Búsqueda Local vs Algoritmos Genéticos

Dataset	BL	Mejor AG	Mejor AM	Δ(BL−AG)
ibex_35	−4,491	AGE-BLX (−4,521)	AM-Best (−4,503)	+0,030
syp_100	−2,791	AGE-BLX (−2,968)	AM-Best (−3,079)	+0,177
syp_500	−2,628	AGG-BLX (−2,871)	AM-Best (−2,933)	+0,243

BL domina globalmente: el gap crece con N (de 0,030 a 0,243). Causas: (1) el operador de movimiento es *problem-tailored*, más eficiente que cruce/mutación genérica; (2) BL invierte las 10000 evals en una única trayectoria de mejora monotónica; (3) en ibex_35 converge en solo $\approx$ 2100 evals, lo que indica un paisaje de fitness suave y navegable.

6.6. Capacidad de generalización

Dataset	BL (test)	Mejor AG (test)
ibex_35	−3,085	AGE-BLX (−3,102)
syp_100	−3,757	AGE-BLX (−3,777)
syp_500	−3,916	AGG-BLX (−3,924)

BL supera a los AGs también en test, lo que sugiere mayor robustez y mejor generalización. Los movimientos suaves (15 % de peso) producen soluciones que respetan las correlaciones entre activos; cruce y mutación pueden introducir discontinuidades que sobreajustan al periodo de entrenamiento.

6.7. Variabilidad, coste computacional y análisis de boxplots

Algoritmo	ibex_35	syp_100	syp_500
Random	0,024s	0,064s	1,734s
BL	0,001s	0,039s	1,168s
AGG-BLX	0,008s	0,037s	0,673s
AGE-BLX	0,008s	0,034s	0,663s
AM-Best	0,016s	0,133s	1,326s

En syp_500, los AGs puros (0,65–0,67s) son los más rápidos porque no incurren en overhead de BL. AM-All es el más lento (2.11s) por aplicar BL intensiva a $\approx$ 90 individuos. Random (1.73s) es lento al generar 10000 soluciones completas.

Análisis de los boxplots (Figuras 4–6):

- **Greedy**: línea única sin varianza (determinístico) en los tres datasets. En ibex_35 ofrece el mejor valor medio, pero en syp_100 y syp_500 es el peor algoritmo, lo que refleja que la solución voraz colapsa cuando N crece.
- **Random**: cajas anchas con bigotes largos y outliers frecuentes, confirmando la alta varianza. En ibex_35 aparecen outliers superiores (semillas que por azar aciertan buenas combinaciones).
- **BL**: caja muy compacta en todos los datasets, especialmente en ibex_35 donde converge siempre en el mismo óptimo local ($\approx$ 2100 evals). La consistencia aumenta con N, confirmando que el operador de movimiento produce trayectorias estables.

- **AGs**: varianza baja y homogénea entre variantes, gracias al efecto de promediado de la población. BLX tiene medianas ligeramente superiores a Aritmético en todos los datasets, y AGE supera a AGG en ibex_35 y syp_100.
- **AM-All**: varianza visiblemente mayor que AM-Rand y AM-Best (especialmente en syp_100 y syp_500), reflejando que el agotamiento prematuro del presupuesto hace que el resultado dependa más de la semilla inicial.
- **AM-Rand y AM-Best**: distribuciones similares a los AGs puros en varianza, con medianas ligeramente superiores, lo que confirma que la BL selectiva mejora la calidad media sin aumentar la variabilidad.

6.8. Conclusiones generales

1. **BLX supera a Aritmético** en ambos esquemas: la exploración fuera del convex hull es crítica.
2. **AGE supera a AGG** con presupuesto limitado: la élite implícita es más eficiente que el reemplazo generacional.
3. **AM-All fracasa** al concentrar la BL en fases tempranas con población inmadura. AM-Best y AM-Rand son competitivos pero no dominantes frente a BL.
4. **BL domina globalmente**: operador problem-tailored y convergencia monotónica son decisivos; el gap respecto a los AGs crece con N.
5. Los **boxplots** confirman los resultados medios: BL es el más consistente, Random el más disperso, y AM-All el AM con mayor varianza.